Highlights

RPFUZZ: Efficient Network Service Fuzzing via Pruning Redundant Mutation

Wenfeng Lin, Fangliang Xu†, Zhiyuan Jiang, Gang Yang, Zhiwei Li, Chaojing Tang

- We propose redundant mutation pruning technique for network service fuzzing. By pruning mutated suffix packet sequence which is non-coverage-improving, network service fuzzer can achieve higher throughout. This is achieved by redundant mutation oracle, which leverage code coverage and identified service-related variables' (SRVs) value to decide whether continuing current execution is advisable.

- To precisely identify SRVs in network services, we propose an identification method combining with call stack analysis and value flow graph analysis. This method is based on SRV's programming features, which can be applied in various network service.

- Based on technique above, We implement RPFUZZ. RPFUZZ achieved more than 185.92% (average 56.39%) throughput enhancement over NYX-NET, while improving maximum 4.27% code coverage (average +1.02%). It successfully identified 1753 unique crashes across 6 targets without ASAN and a buffer overflow vulnerability in ProFTPD (assigned CVE-2024-57392)

RPFUZZ: Efficient Network Service Fuzzing via Pruning Redundant Mutation

Wenfeng Lin, Fangliang Xu[†], Zhiyuan Jiang, Gang Yang, Zhiwei Li, Chaojing Tang

[a]*College of Electronic Science and Technology, College of Computer Science and Technology, College of Information and Communication, National University of Defense Technology, Changsha, 410005, Hunan, China*

Abstract

Coverage-guided fuzzing (CGF) has proven its outstanding performance on vulnerability detection. However, existing approaches exhibit limitations when handling network service. Restricted by network I/O duration and chronology, long packet sequences crafted by fuzzers incur a substantial execution cost. Test cases with such non-coverage-improving mutations (i.e. redundant mutation) can significantly reduce fuzzing throughput and compromise vulnerability discovery.

To address this issue, we propose RPFUZZ, a novel network fuzzing framework designed to systematically reduce redundant mutations: (1) We propose redundant mutation pruning for network service fuzzing. By early terminating redundant mutations' execution, RPFUZZ can achieve higher throughput. (2) To detect redundant mutation, we propose redundant mutation oracle. This oracle dynamically judges whether a test case is redundant according to current code coverage and value of service-related variables (SRVs). (3)To identify SRVs, we propose an integrated approach combining dynamic call stack analysis with static value-flow graph (VFG) analysis.

To evaluate the performance of RPFUZZ, we implement a prototype on top of NYX-NET. We conduct thorough experiments on ProFuzzBench, a benchmark that consists of 12 real-world network services. The results indicate that RPFUZZ achieves over 185% improvement in throughput and 1.02% rise in code coverage compared with NYX-NET. Besides, RPFUZZ has successfully uncovered 1753 unique crashes across 6 network services, including an unreported vulnerability (assigned to CVE-2024-57392) in ProFTPD, which has been well tested.

Keywords:
Fuzzing, Network Service, Network Protocol, Security, Software Testing

1. Introduction

Network services, as critical components of computer communication, inherently face amplified security risks due to their reliance on network I/O for data exchange (crowd strike (2025)). Vulnerabilities in these software often enable remote exploitation through maliciously crafted packets, leading to severe consequences such as information leakage and unauthorized code execution. Notable examples include the HeartBleed vulnerability (CVE-2014-0160) (NVD (2025a)) in OpenSSL, which exposed sensitive user authentication data, and the SMBGhost flaw (CVE-2020-0796) (NVD (2025b)) in Microsoft's Server Message Block protocol, permitting remote code execution (RCE). These incidents underscore the urgent need for robust vulnerability discovery mechanisms tailored to network services.

Coverage-guided fuzzing (CGF) has demonstrated remarkable effectiveness in software vulnerability discovery (antonio morales (2020); libfuzzer (2025)). Consequently, pioneering studies such as AFLNET have introduced coverage-guided techniques into network service fuzzing (Pham et al. (2020)), specifically tackling the limitations of template-based fuzzer in exploring code paths beyond predefined templates (peach (2025); boofuzz (2024)).

CGF requires multiple test cases as initial seeds. As shown in Figure 1, a test case is a network packet sequence in network service fuzzing. When conducting mutation, the test case will be divided into three parts: the prefix sequence ($M1$) remains unmutated and is used to drive target into a specific state; the candidate subsequence ($M2$) undergo mutation ($M2'$) to explore the code space or vulnerabilities. To avoid potentially missing code that may require additional packets to be triggered, suffix sequence ($M3$) is further sent. However, as shown in Figure 2, this paradigm will introduce **redundant mutation** (i.e. mutated test case without new coverage). Existing research revealed that only a small fraction of mutations contribute to

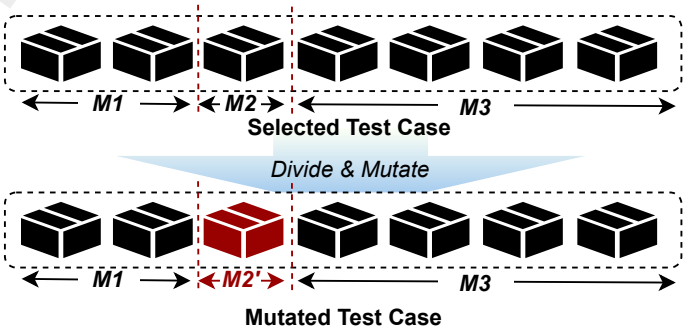

Figure 1: Typical mutation strategy in modern coverage-guided network service fuzzing

new coverage (Nagy and Hicks (2019); Yue et al. (2020)). While the $M2' + M3$ can effectively explore new code in the early stages, as fuzzing process continues, the chance of activating new code through mutation steadily declines. What's worse, this approach incrementally generates long sequences, which further degrades network service fuzzing. This degradation becomes particularly pronounced in network fuzzing where I/O operations are inherently slow.

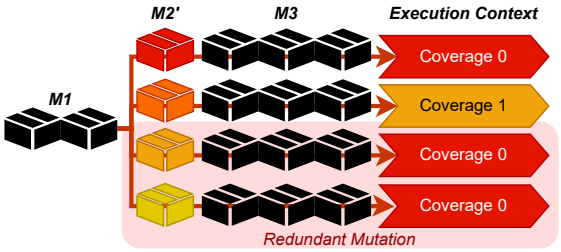

Figure 2: Illustration of redundant mutation phenomenon in network service fuzzing

Numerous studies have proposed solutions from various perspectives to mitigate this issue: ① **Fast customized network I/O**(Qin et al. (2023); Fu et al. (2023)): These work directly increase packet throughout by modifying network I/O. ② **Snapshot mechanism**(Schumilo et al. (2022); Li et al. (2022)): They record and restore the program execution context (e.g. registers, memory, file descriptors, etc.) after receives $M1$ through snapshot mechanisms, avoiding the time overhead introduced by initialization and receiving $M1$ for each iteration. ③ **Shorter seeds scheduling**(Liu et al. (2022)): Heuristic algorithms are used to select shorter test cases that have a higher probability of triggering new code. However, these approaches still require executing complete test cases that may not bring new coverage. Avoiding redundant execution can significantly improve fuzzing throughout. Therefore, a natural approach to further enhance throughput is dynamically deciding whether continuing current execution is advisable. To achieve this, the core challenge lies in how to effectively make this decision.

In this work, we introduce **redundant mutation pruning** technique to address the aforementioned issue. As Section 2 will show, by merely analyzing and determining whether the mutated $M2'$ is redundant, this technique can save the time required for sending $M3$, thereby further enhancing the efficiency of fuzzing for network services.

In summary, we make the following main contributions:

1. We implement **RPFUZZ**, a coverage-guided network fuzzing framework leveraging redundant mutation oracle to boost fuzzing throughout.
2. We implement **Redundant Mutation Oracle**, the first approach to evaluates potential value of ongoing mutation based on coverage and service-related variables.
3. We implement technique combining dynamic call stack analysis with static value-flow graph (VFG) analysis to identify **service-related variable** (SRV),
4. In our evaluation, we show that RPFUZZ can achieve more than 185% throughput (average +56.39%) and 4.27% coverage (average +1.02%) improvement compared with state-of-the-art fuzzing tool NYX-NET on ProFuzzBench. Furthermore, RPFUZZ found 1753 unique crashes and an unreported CVE in ProFTPD.

2. Motivating Example

The file deletion workflow in ProFTPD exemplifies a critical challenge in stateful service fuzzing. As shown in Listing 1, to successfully delete a file via the `DELE` command, the server enforces a sequence of interdependent validations spanning multiple client-server interactions. For instance, a client must first authenticate (`USER/PASS`)($M1$), navigate to a permitted directory (`CWD`)($M2$), and ensure the target path adheres to allow/deny filters—all before issuing `DELE` ($M3$). Each step establishes server-side state (e.g. authenticated user identity, working directory) that subsequent commands rely on.

Fuzzing tools that naively mutate individual packets, however, risk disrupting these prerequisites. A single malformed `CWD` command (e.g. an invalid path mutation)($M2'$) leaves the server in a state where even a perfectly crafted `DELE` packet cannot trigger the `unlink()` system call, as directory permissions or path validation checks fail early. This leads to redundant mutation: mutated $M2'$ that corrupt precondition-establishing packets (e.g., authentication tokens, path context) inevitably prevent $M3$ from reaching deep code paths.

If redundant mutation in $M2'$ can be dynamically identified during fuzzing iterations and preemptively terminated, $M3$ transmissions could be abandoned to avoid redundant executions.

Listing 1: Motivating Example in ProFTPD `core_dele` Function with Multi-Step Validation

```
MODRET core_dele(cmd_rec *cmd) {
  int res;
  // ...
  // Step 1:  Decode and sanitize client-provided path
  decoded_path = pr_fs_decode_path2(cmd->tmp_pool, cmd->
      ↪ arg,
  FSIO_DECODE_FL_TELL_ERRORS);
  // Step 2:  Validate path against allow/deny filters
  res = pr_filter_allow_path(CURRENT_CONF, path);
  switch (res) { /* ... */ }
  // Step 3:  Convert to canonical absolute path
  path = dir_canonical_path(cmd->tmp_pool, path);
  if (path == NULL) { /* ... */ }
  // Step 4:  Verify directory write permissions
  if (!dir_check(cmd->tmp_pool, cmd, cmd->group, path,
      ↪ NULL))  { /* ... */ }
  // Step 5:  Check file metadata (existence/type)
  res = pr_fsio_lstat_with_error(cmd->tmp_pool, path, &
      ↪ st, &err);
  if (res < 0) { /* ... */ }
  // Step 6:  Delete file via unlink() system call
  res = pr_fsio_unlink_with_error(cmd->pool, path, &err)
      ↪ ;
  // ...

}
```

3. Preliminary

3.1. Snapshot-powered Network Service Fuzzing

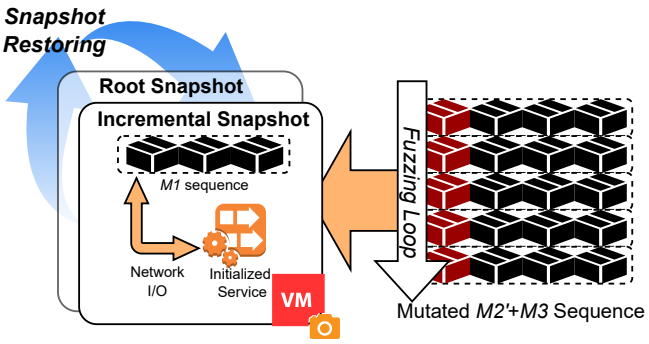

Figure 3: Overview of snapshot mechanism in NYX-NET. Root snapshot is taken after service is initialized. Incremental snapshot is taken after $M1$ sequence is received. One fuzzing iteration is performed based on incremental snapshot

For CGF in network services, each mutation cycle requires the service under test to complete initialization and receive $M1$ to drive it into a particular state. As shown in figure 3, the snapshot mechanism mitigates this overhead by saving the copy of memory and registers of the target program when it reaches a particular state. Then, it can quickly restore the program to specific state whenever needed. NYX-NET (Schumilo et al. (2022)) further introduces incremental snapshots, which reduce the cost of taking further snapshot by recording and restoring only the memory pages that differ from root snapshot (i.e. dirty pages). However, after a snapshot is restored, the target still needs to receive mutated packet sequences through network I/O. The problem of redundant mutations prevents NYX-NET from fully leveraging the potential of incremental snapshots to enhance testing efficiency. In this paper, we employ redundant mutation pruning techniques to reduce the transmission of non-coverage-increasing mutation sequences over network I/O, therefore optimize the frequency of snapshot restoration, and further enhance the throughput.

3.2. Program Value Flow Graph (VFG)

The Value-Flow Graph (VFG) is a graphical representation of a program that systematically traces the propagation of values and dependency among variables. Specifically, VFG models define-use chains of variables through nodes and edges. The nodes in VFG encompass information such as variable definitions, memory operations, control flow transitions, and numeric calculations. The edges in VFG, on the other hand, establish data flow dependencies among multiple variables through inter-procedural pointer analysis, which includes pointer aliasing, function calls, and assignment propagation. By analyzing VFG, researchers can infer the runtime behavior of variables without executing the target program. VFG analysis has been widely applied in security scenarios, such as memory leak detection, hybrid fuzzing, and directed grey-box fuzzing. (Sui et al. (2014); Chen et al. (2020, 2018)).

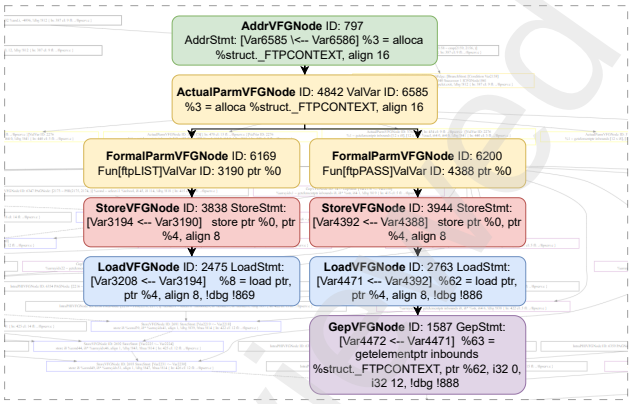

Figure 4: A snippet of identified service related variable (SRV) `FTPCONTEXT.access` 's propagation on value flow graph (VFG) in Lightftp

As shown in figure 4, we leverage VFG analysis and propose specific rules to identify service-related variables (SRVs) by tracing value propagation.

3.3. Hidden Markov Process (HMP)

A Hidden Markov Process (HMP) is a probabilistic model that describes the temporal dependencies of operation sequences through an underlying Markov chain and emission probabilities associated with specific states (Dymarski (2011)). HMP is particularly effective in characterizing network service behaviors due to its intrinsic alignment with network services:(1) stateful transition logic (i.e. protocol specifications of network services), (2) observable event sequences (i.e. information such as code coverage, service responses, and SRV values), and (3) the implicit protocol logic governing valid input-output relationships. HMP has been utilized in network traffic analysis and protocol reverse engineering (Whalen et al. (2010); Gascon et al. (2015); Li et al. (2024)). Theses facts form the basis of our insight: packet sequences drive the service into different states, and further testing under invalid state transitions is unlikely to trigger deep code. We propose redundancy-pruning testing, achieving fuzzing efficiency through context-aware resource reallocation while maintaining vulnerability discovery capability.

4. Design

This section introduces technical details for redundant mutation pruning fuzz (RPFUZZ) and how we design the redundant mutation oracle.

Figure 5 presents the high-level framework of RPFUZZ. The core design can be summarized as follows: During the fuzzing cycle, the redundancy oracle collects grey-box metrics (code coverage and SRV vaules) from the following execution of: ① the $M1$ sequence,② the $M2$ sequence, and ③ its mutated variant $M2'$. By leveraging a systematical analysis of these metrics, the redundant mutation oracle dynamically determines whether to execute the $M3$ sequence.

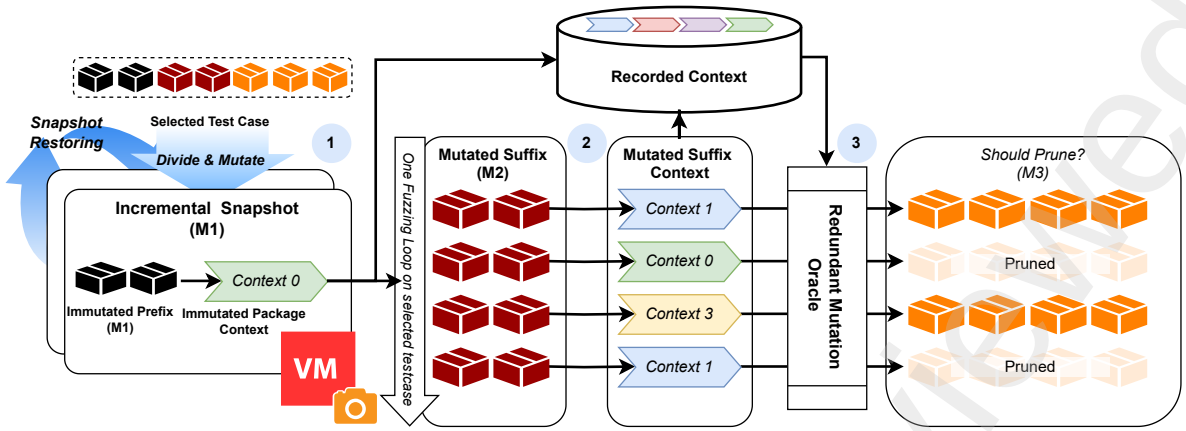

Figure 5: Workflow of Redundant Mutation Pruning Fuzz

Taking the example in Section 2, after completing user authentication via `USER/PASS` sequence ($M1$), RPFUZZ captures an incremental snapshot. The redundancy oracle then performs systematic analysis on both the original packet `CWD` ($M2$) and its mutation ($M2'$)'s execution context on code coverage metrics and SRVs' values. Through this framework, RPFUZZ intelligently determines whether subsequent execution of `DELE` ($M3$) should continue. This ensures `DELE` is triggered exclusively when novel execution contexts emerge, effectively reducing redundant test from $M3$.

To achieve this, we initially identify service-related variables (SRVs) via VFG analysis (Section 4.1). The runtime value of SRVs, in conjunction with code coverage, is fed into the mutation redundancy oracle for real-time redundant execution detection (Section 4.2). Finally, guided by the oracle, adaptive pruning dynamically alternates between regular and pruning fuzz to efficiently explore the code space while eliminating redundant executions (Section 4.3).

4.1. Service-Related Variable (SRV) Identification

Code coverage metrics offer incomplete contextual awareness for redundancy detection in network protocol fuzzing (Fioraldi et al. (2021); Aschermann et al. (2020)). To address this limitation, we introduce service-related variables (SRVs) - program variables that encode protocol state transitions through their value changes. Our empirical analysis of 10 protocol implementations (including FTP, SMTP, and TLS) reveals that SRVs are widely used in network services. Based on our observation, we conclude SRVs programming features as follow:

1. *Specification Definition:* SRVs are explicitly declared by developers for implementing protocol specifications. As shown in Listing 2, LightFTP utilizes structure field `Access` to describe the verification of user identity and permissions as specified in the FTP standard RFC959. Its definition is independent of external system components such as network socket.
2. *Extended Lifetime:* SRVs are Reside in global variables or function local-variables persisting across multiple network transactions. As shown in Listing 2, during the communication process between LightFTP and the client, Function `ftp_client_thread` remains alive throughout. `ctx`, which is a structure used to describe the interaction state, needs to persist throughout the entire communication process, and therefore it is defined and used within `ftp_client_thread`.
3. *Control-Flow Critical:* SRVs directly influence conditional branches governing protocol state transitions. As shown in Listing 2, function `ftpMKD` in LightFTP leverages variable `Access` to control permission of making new directory. Users with access permission `FTP_ACCESS_NOT_LOGGED_IN` are unable to create new directories using ftp command `MKD`.
4. *Non-Computational:* SRVs never participate in arithmetic operations (+, ×, etc.). As shown in Listing 2, The value of SRV can also change, but such changes typically do not result from specific operators like addition and multiplication. These operations are more likely related to loop control like buffer reading. This phenomenon is associated with the programming habits of network service designers. Most programmers are more accustomed to using specific value assignments to describe the state of protocol.
5. *Input-Sensitive:* SRVs exhibit value changes traceable to network input through data/control dependencies. As shown in Listing 2, `Access` changes in function `ftpPASS` based on user input. Depending on the differences in USER and PASS commands, the access variable can be assigned various values, each corresponding to different path access permissions levels.

Automatically identifying SRV is a challenging task. As demonstrated in Section 3.2, the program value flow graph (VFG) is an effective method for statically describing the programming feature of variables. Based on these characteristics, we extract SRV by analyzing the propagation of all variables in VFG (as shown in Figure 4). However, static analysis methods inevitably produce false positives, and excessive false positives can significantly increase the execution overhead for SRV instrumentation. To address this issue, we employ dynamic function call stack analysis to identify long-live functions. In

Listing 2: Five programming features of service related variable (SRV) in motivating example SRV `ctx.Access` in LightFTP

```
Feature 1:  Specification Definition:
typedef struct  _FTPCONTEXT {
  // ...
  int          Access;
  char         CurrentDir[PATH_MAX];
  // ...
} FTPCONTEXT, *PFTPCONTEXT;

Feature 2:  Extended Lifetime
void *ftp_client_thread(SOCKET *s)
{
  FTPCONTEXT ctx __attribute__ ((aligned(16)));
  // ...
  while (getsockname(ctx.ControlSocket, (struct sockaddr
      ↪  *)&laddr, &asz) == 0 )
  {
    // ...
    rv = ftpprocs[c](&ctx, params);
    break;
  }
  // ...
}

Feature 3:  Control-Flow Critical
int ftpMKD(PFTPCONTEXT context, const char *params)
{
  // ...
  if ( context->Access == FTP_ACCESS_NOT_LOGGED_IN )
    return sendstring(context, error530);
  // ...
}
Feature 4:  Input Sensitive
Feature 5:  Non-computational
int ftpPASS(PFTPCONTEXT context, const char *params)
{
  // ...
  context->Access = FTP_ACCESS_NOT_LOGGED_IN;
  do {
    if (strcasecmp(temptext, "admin") == 0 ){
      context->Access = FTP_ACCESS_FULL;
      break;
    }
    // ...
  }
}
```

this way, we can not only reduce the overhead of static analysis, but also minimize false positives caused by local variables from non-long-live functions. Above all, our SRV identification method uniquely combines lightweight dynamic call stack profiling and context-sensitive static value-flow graph analysis (as shown in Figure.6). Based on proposed feature, we design heuristic filtering rules to recognize SRVs before fuzzing.

4.1.1. SRV Definition Site Filtering by Call Stack Analysis

To minimize static analysis overhead, we constrain variable candidates using feature 1-2:

Long-lived functions identification: We instrument the network I/O functions (such as the send/recv family) of the target program, and then capture complete call stacks during live protocol sessions. We identify long-lived communication functions that maintain stack presence across more than 3 consecutive network transactions. The result will reduce analysis overhead in reference analysis.

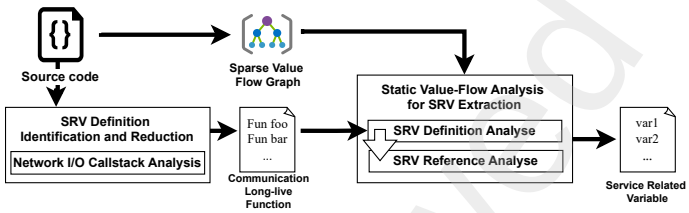

Figure 6: Workflow of Service-related Variable(SRV) Identification

SRV definition site filtering: We take the following SRV candidates, which comprise: ① All global variables in the target service ② Local variables in long-lived communication functions ③ Structure fields accessed through these variables

4.1.2. SRV Identification by VFG Analysis

By dynamically analyzing and filtering potential SRV candidates, we conduct context-sensitive taint propagation on field-sensitive sparse value-flow graphs for precise SRV identification. Our analysis incorporates four specialized propagation rules to handle both primitive variables and structure fields, as formalized in Algorithm 1 and Algorithm 2:

1. *Field-sensitive Node Propagation Rules:* For structure fields accessed via `GetElementPtr` instructions, we:
   (1) Initiate forward propagation from structure base `Allocation` node in VFG.
   (2) Suspend analysis for variables with dynamic `GetElementPtr` offsets. Treat and classify each unique `GetElementPtr` node (identified by `<base_alloca, Gep_offset>` tuple) as an independent SRV candidate.
   (3) Terminate propagation at the following VFG Node: `Arithmetic`, `Comparison`, `Phi`, `Branch`, `ParameterOut`, and node related to system API calls(e.g. `recv()`, `poll()`). These rules are introduced in `forwardSpreadRulesFindGep` in Algorithm 1
2. *Dereference Node Propagation Rules:* We implement different logic for structure fields versus primitive variables dereference propogation, which is demonstrated as function `forwardSpreadRulesFindLoadStore` :
   (A) Scalar Variable: We first initiate direct propagation from variable allocation points, and then enforce consistent termination criteria: (1) Stop at nodes such `Arithmetic`, `Phi`, `Comparison`, `Branch` (2) Discontinue at `Parameter` node related to system API
   (B) Structure Field:, we perform GetElementPtr-based forward analysis from allocation nodes, track and classify value flows through specific field offsets. We additionally apply termination conditions identical to `GetElementPtr` node.
3. *Load Node Dereference Analysis:* Guided by the dereference features of service-related Variables (SRVs), we architect specialized propagation rules for their associated `Load` nodes within the value-flow graph. In detail, we trace load nodes through constraints defined in Table 1.

Algorithm 1: Service-related Variable Extraction (For Structure Variable's Field)

Input: Target Sparse Value Flow Graph G, Communication long live Function Set F
Output: service-related Variables SRV

$WorkList \leftarrow \emptyset$;
foreach $f \in F$ **do**
  $struct\_alloc\_node \leftarrow$ `GetSturctAllocationNode`(G, f);
  $WorkList.append(struct\_alloc\_node)$;
  while $WorkList \neq \emptyset$ **do**
    $cur\_node \leftarrow WorkList.pop()$;
    $post\_node \leftarrow$ `GetPostNode`(G, cur_node);
    if `forwardSpreadRulesFindGep`$(post\_node)$ **then**
      $WorkList.append(post\_node)$;
foreach $(struct\_alloc\_node, GEP\_id) \in GEP\_SET$ **do**
  foreach $Gep\_Node \in GepNodeList[GEP\_id]$ **do**
    $WorkList.append(Gep\_Node)$;
    while $WorkList \neq \emptyset$ **do**
      $cur\_node \leftarrow WorkList.pop()$;
      $post\_node \leftarrow$ `GetPostNode`(G, cur_node);
      if `isLoadNode`(cur_node) **then**
        if `load_to_branch`(cur_node) **then**
          $num\_branch\_load \leftarrow num\_branch\_load + 1$;
        if `load_to_bin_op`(cur_node) **then**
          $num\_bin\_op\_load \leftarrow num\_bin\_op\_load + 1$;
        if `load_from_user`(cur_node) **then**
          $num\_user\_load \leftarrow num\_user\_load + 1$;
      else if `isStoreNode`(cur_node) **then**
        if `store_to_branch`(cur_node) **then**
          $num\_branch\_store \leftarrow num\_branch\_store + 1$;
        if `store_to_bin_op`(cur_node) **then**
          $num\_bin\_op\_store \leftarrow num\_bin\_op\_store + 1$;
        if `store_from_user`(cur_node) **then**
          $num\_user\_store \leftarrow num\_user\_store + 1$;
        if `store_multi`(cur_node) **then**
          if `isStoreConstant`(cur_node) **then**
            $num\_multi\_store \leftarrow num\_multi\_store + 1$;
          else
            $num\_multi\_store \leftarrow num\_multi\_store + 2$;
      if `forwardSpreadRulesFindLoadStore`$(post\_node)$ **then**
        $WorkList.append(post\_node)$;
    if $num\_branch\_load > 0 \wedge num\_bin\_op\_load = 0 \wedge num\_user\_load = 0 \wedge num\_branch\_store > 0 \wedge num\_bin\_op\_store = 0 \wedge num\_user\_store = 0 \wedge num\_multi\_store > 1$ **then**
      $SRV.append((struct\_alloc\_node, GEP\_id))$;

Algorithm 2: Service-related Variable Extraction (For Scalar Variable)

Input: Target Sparse Value Flow Graph G, Communication long live Function Set F
Output: service-related Variables SRV

$WorkList \leftarrow \emptyset$;
foreach $f \in F$ **do**
  $var\_alloc\_node \leftarrow$ `GetVarAllocationNode`(G, f);
  $var\_list.append(var\_alloc\_node)$;
foreach $var\_alloc\_node \in var\_list[GEP\_id]$ **do**
  $WorkList.append(var\_alloc\_node)$;
  while $WorkList \neq \emptyset$ **do**
    $cur\_node \leftarrow WorkList.pop()$;
    $post\_node \leftarrow$ `GetPostNode`(G, cur_node);
    if `isLoadNode`(cur_node) **then**
      if `load_to_branch`(cur_node) **then**
        $num\_branch\_load \leftarrow num\_branch\_load + 1$;
      if `load_to_bin_op`(cur_node) **then**
        $num\_bin\_op\_load \leftarrow num\_bin\_op\_load + 1$;
      if `load_from_user`(cur_node) **then**
        $num\_user\_load \leftarrow num\_user\_load + 1$;
    else if `isStoreNode`(cur_node) **then**
      if `store_to_branch`(cur_node) **then**
        $num\_branch\_store \leftarrow num\_branch\_store + 1$;
      if `store_to_bin_op`(cur_node) **then**
        $num\_bin\_op\_store \leftarrow num\_bin\_op\_store + 1$;
      if `store_from_user`(cur_node) **then**
        $num\_user\_store \leftarrow num\_user\_store + 1$;
      if `store_multi`(cur_node) **then**
        if `isStoreConstant`(cur_node) **then**
          $num\_multi\_store \leftarrow num\_multi\_store + 1$;
        else
          $num\_multi\_store \leftarrow num\_multi\_store + 2$;
    if `forwardSpreadRulesFindLoadStore`$(post\_node)$ **then**
      $WorkList.append(post\_node)$;
  if $num\_branch\_load > 0 \wedge num\_bin\_op\_load = 0 \wedge num\_user\_load = 0 \wedge num\_branch\_store > 0 \wedge num\_bin\_op\_store = 0 \wedge num\_user\_store = 0 \wedge num\_multi\_store > 1$ **then**
    $SRV.append(var\_alloc\_node)$;

4. *Store Node Dereference Analysis:* When propogate to `Store` VFG nodes, different from `Load` nodes, we enforce the following rules in Table 2.

The SRV identification methodology establishes a protocol-aware program analysis framework that effectively integrates dynamic execution profiling with context-sensitive static value-flow analysis. This technique effectively overcomes the limitation of context blindness in conventional code coverage metrics, providing a rigorous foundation for the redundancy detection mechanism described in Section 4.2.

4.2. Redundant Mutation Oracle

The Redundant Mutation Oracle forms the cornerstone of our fuzzing framework. As shown in Figure 7, this module integrates SRV value patterns with code coverage information to enhance the accuracy of execution fingerprinting, thereby enabling the precise identification of duplicate executions via global hash table lookups.

4.2.1. SRV Runtime Value Feedback

The instrumentation framework injects probe code at `Store` operations of SRVs identified in Section 3.1. These probes dynamically capture the runtime values and the corresponding IDs of the monitored SRVs and store them in shared memory using an AFL-inspired bitmap mechanism, providing enhanced feedback for fuzzing campaigns. The implementation adopts type-specific instrumentation policies based on SRVs' type:

For primitive-type SRVs, we inject common concrete value-recording probes.

For pointer-type SRVs, we implement null pointer check abstraction.

This design mitigates the state explosion problem by encoding validity states as binary flags, which eliminates memory layout dependencies through address abstraction. These instrumentation will serve as additional feedback for the redundancy oracle, making the decision more accurate.

Feature	VFG Rule	Rule Description
Specification definition	`load_user`	Backward propagate from the `load` node. If the `load` node is passed as an actual parameter to a function call, and the called function is a system function, then the condition is **false**.
Control Flow Critical	`load_to_branch`	Forward propagate from the `load` node. If there exists a `cmp` (compare) node after the `store` node, and a `branch` node after the `cmp` node, then the condition is **true**.
Non-Computational	`load_to_bin_op`	Forward propagate from the `load` node. If there exists a binary operation node after the `store` node, and the binary operation is one of {addition, subtraction, multiplication, division, modulo}, then the condition is **false**.

Table 1: Load Dereference Analysis Rules for service-related Variable extraction algorithm

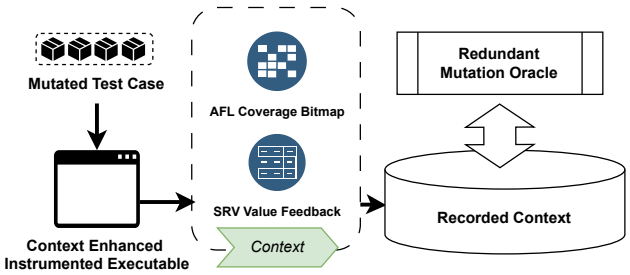

Figure 7: Design of Mutation Redundancy Oracle

4.2.2. Redundant Mutation Decision

The oracle's decision logic operates as a stateful finite automaton,by combining real-time fingerprint matching and dynamic registry updates. Upon the completion of a test case t, the oracle performs the following sequence:

Step 1: Composite Fingerprint Generation: For each test case execution t, we compute a fingerprint through:

$$\Phi(t) = (\text{Hash}(\mathcal{B}_{COV}), \text{Hash}(\mathcal{B}_{SRV})) \tag{1}$$

where $\mathcal{B}_{COV}$ is current runtime coverage bitmap after receiving packages in AFL-style[1]; $\mathcal{B}_{SRV}$ is SRV value bitmap constructed in 4.2.1.

Step 2: Historical Context Query: During the fuzzing process, we maintain a hierarchical storage architecture for efficient fingerprint matching. The oracle determines whether the execution is redundant based on the following formal criteria:

$$O(t) = \begin{cases} True & \text{if } \exists t' \in \mathcal{H} : \Phi(t) = \Phi(t') \\ False & \text{otherwise} \end{cases} \tag{2}$$

Step 3: Global Context Registry Update: Every time when encounter with new context fingerprint, it will be registered in the set of recorded contexts:

$$\mathcal{H} \leftarrow \mathcal{H} \cup \{\Phi(t)\} \tag{3}$$

In this section, the oracle deterministically identifies redundant executions through SRV-enhanced context, combining code coverage hashes and CRV value patterns. Leveraging a historical context knowledge, the oracle queries global fingerprints and dynamically updates the registry for new executions.

[1]It should be noted that the bitmap mentioned here does not refer to the comprehensive bitmap recorded during AFL's full fuzzing routine, but rather to the bitmap specifically generated upon completion of the current test case. It is different from bitmap used in coverage statistics.

Feature	VFG Rule	Rule Description
Specification definition	`store_user`	Backward propagate from the `store` node. If there is an actual return outgoing node before the `store` node, and the node represents a system function, then the condition is **false**.
Control Flow Critical	`store_to_phi`	Forward propagate from the `store` node. If a `phi` node exists after the `store` node, then the condition is **true**.
Non-Computational	`store_to_bin_op`	Backward propagate from the `store` node. If a binary operation node exists after the `store` node, and the binary operation is one of {addition, subtraction, multiplication, division, modulo}, then the condition is **false**.
Input-Sensitive	`store_multi`	The `store` node itself can store variables or constants. Constants count as 1 point, variables count as 2 points. If the score for multiple values being changeable is greater than 1, then the condition is **true**.

Table 2: Store Dereference Analysis Rules for service-related Variable extraction algorithm

4.3. Adaptive Redundancy Pruning

Stateful network services impose temporal dependencies, necessitating specific packet sequences to reach vulnerable states. Interactions with network services can be formally modeled as a Hidden Markov Process (HMP), where execution contexts saturated with redundancy exhibit diminishing probabilities of discovering novel context. Our key insight: **suffix mutations in redundant contexts exhibit diminishing coverage reward.** We therefore develop an adaptive suffix pruning mechanism that strategically allocates testing budget to high-potential sequences through tight integration with the redundancy oracle in section 4.2.

4.3.1. Redundant Mutation Pruning

The redundant mutation pruning fuzz is performed through three-stage execution analysis, as formalized in Algorithm 3. Figure 5 also shows the core workflow as follows:

Stage 1: Historical Snapshot selection: In this stage, RPFUZZ divides selected test case into prefix and suffix by considering history snapshot position (line 1-4 in Algorithm.28). Given an input test case $t = \langle p_1, ..., p_n \rangle$:

1. If the test case was discovered through mutation when a snapshot was captured at the k packet, we randomly select a split position within $\langle P_k, ..., P_n \rangle$ to divide the sequence

into prefix $m_1 = \langle P_1, ..., P_k \rangle$ and suffix $\langle P_{k+1}, ..., P_n \rangle$, where k is uniformly sampled from $[k, n]$.

2. Capture current snapshot via `Run&Snap`($program, m_1$)

Stage 2: Suffix Segmentation: RPFUZZ will further divide suffix by redundancy observation window size τ_j and length threshold τ_p(τ_j is 25% length of τ_p)(line 4-7 in Algorithm.28)

1. If the suffix $\langle P_{k+1}, ..., P_n \rangle$ length is larger than threshold τ_p, suffix is further split into redundancy observing sequence $m_2 = \langle P_{k+1}, ..., P_j \rangle$ and remaining sequence $m_3 = \langle P_{j+1}, ..., P_n \rangle$ by redundancy observation window τ_j.
2. If suffix is segmented successfully, RPFUZZ will execute immutated m_2 to update current global context registry, and redundancy-probing mutation will be performed in stage 3.
3. If not segmented successfully, RPFUZZ will perform regular fuzz on suffix as NYX-NET. [2]

Stage 3: Redundancy Oracle Guided Suffix Pruning: In pruning fuzz process, RPFUZZ will decide whether early terminate execution of full suffix by query mutation-redundancy oracle (line 8-26 in Algorithm.28). For each mutation iteration before running out fuzzing energy:

1. Execute mutated observing sequence $m_2^{'}$ and captured current context $Ctx_2^{'}$.
2. Update corpus based on execution status if an interesting (new code overage or crash) mutated test case is found.
3. Query mutation redundancy oracle. If true, discard following execution of m_3.
4. If false, execute full sequence and update global context registry with $Ctx_2^{'}$. Update corpus based on execution status if an interesting mutated test case is found.

Algorithm 3: One Fuzzing Cycle for Redundancy Pruning Fuzz

Input: Redandunccy Observation Window τ_j, Pruning Length Threshold τ_p, New Coverage Corpus $\mathbb{C}_{Cov}$, Timeout Corpus $\mathbb{C}_{Tiemout}$, Crash Corpus $\mathbb{C}_{Crash}$
Output: Crash Corpus $\mathbb{C}_{Crash}$

```
1  test_case ← Select(C_Cov, C_Tiemout, C_Crash);
2  < m1, suffix > ← HistorySnapshotSelection(test_case);
3  Snapshot ← Run&Snap(prog, m1) ;
4  if Should_Perform_Truncation(suffix, τp) then
5      < m2, m3 > ← Truncation(suffix, τj) ;
6      (Ctx2, Cov2) ← Run_Snapshot(Snapshot, m2) ;
7      Update_Ctx((Ctx2, Cov2));
8      while Energy > 0 do
9          Energy ← Energy − 1;
10         m2' ← Mutate(m2) ;
11         (Ctx2', Cov2') ← Run_Snapshot(Snapshot, m2') ;
12         if Crash() then
13             C_Crash ← < m1, m2' >
14         else if TimeOut() then
15             C_Tiemout ← < m1, m2' >
16         else if is_Interesting(Cov2') then
17             C_Cov ← < m1, m2' >
18         else if is_Redundant((Ctx2', Cov2')) then
19             Update_Ctx((Ctx2', Cov2'));
20             (Ctx3, Cov3) ← Run_Snapshot(Snapshot, m2' + m3);
21             if Crash() then
22                 C_Crash ← < m1, m2', m3 >
23             else if TimeOut() then
24                 C_Tiemout ← < m1, m2', m3 >
25             else if is_Interesting(Cov3) then
26                 C_Cov ← < m1, m2', m3 >
27 else
28     Perform_Regular_Fuzz(Snapshot, m2)
```

4.3.2. Adaptive Prune Mode Switching

During the initial fuzzing phase where most code paths remain unexplored, random mutation is more effective in discovering new code coverage. However, our empirical analysis reveals that the pruning testing strategy (implemented with a constrained truncation window) exhibits suboptimal efficiency in early-stage coverage improvement. This limitation stems from two inherent characteristics: (1) The reduced-size truncation window restricts the potential for immediate path exploration, and (2) The redundancy detection mechanism's effectiveness depends on sufficient historical execution context that requires gradual accumulation through testing iterations.

To address this phased efficiency disparity, we implement an adaptive dual-mode switching mechanism between **Redundancy Pruning Fuzz** and **Regular Fuzz**. The framework employs dynamically adjusted switching thresholds and follows the operational workflow below:

Step.1 Fuzzing Initialization

- Set independent oscillation parameters C_P, C_R for pruning fuzzing mode and regular fuzzing mode.
- Prioritize regular Fuzz during the bootstrap phase, leveraging its significant advantage in virgin path discovery as established through initial testing.

Step.2 Coverage-Stagnation Monitoring

- Continuously track the discovery of new code coverage during the current fuzzing mode.
- Increase current mode's execution counter when no new coverage is detected per iteration.

Step.3 Adaptive Mode Transition

- Trigger fuzzing mode switching when either counter reaches its oscillation parameters' threshold.
- Reset the activated mode's execution counter, and goto step.2.

In this section, RPFUZZ leverages redundant mutation oracle eliminating unnecessary execution to achieve higher throughout. Adaptive fuzzing mode switching mechanism uses regular fuzzing's exploratory advantages during initial phases while progressively activating pruning fuzzing's efficiency as sufficient execution context accumulates.

[2] We believe there is no need to prune redundant mutation in short test case, since snapshot restoring takes more time than sending one packet.

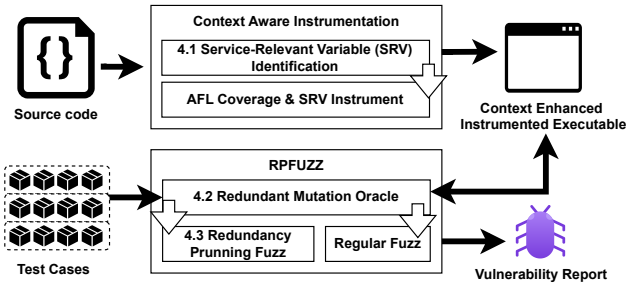

Figure 8: Overview of RPFUZZ framework

5. Implementation

As shown in Figure 8, RPFUZZ consists of two parts: context-aware instrumentation in the pre-fuzz phase and redundancy pruning fuzz in the post-fuzz phase.

We implement context-aware instrumentation on SVF (SVF-tools (2025)) and AFL++ (Fioraldi et al. (2020)). Specifically, we leverage the capabilities of SVF for precise program analysis and AFL++ for its efficient fuzzing framework. By combining these two tools, we can effectively insert instrumentation code that is aware of the program's execution context.

For service-related variable identification, we implement SRV-Extractor utilizing SVF's stride-sensitive pointer analysis and value-flow graphs (Lei and Sui (2019)). This analysis allows us to accurately track the flow of values and pointers in the program, enabling us to identify variables that are relevant to the service's behavior.

For SRV feedback, we extended AFL++'s LLVM mode to insert context enhanced instrumentation. The purpose of this instrumentation is to collect runtime information about the service-related variables during the fuzzing process. This feedback can then be used to guide the redundancy pruning. RPFUZZ is implemented based on the NYX-NET framework (Schumilo et al. (2022)).

6. Evaluation

We evaluate RPFUZZ to answer the following question:

1. Can our SRV extraction method effectively identify service-related Variables (SRVs)? (Section.6.1)
2. What is the execution overhead introduced by SRV instrumentation? (Section.6.2)
3. Can the redundancy oracle effectively reduce the occurrence of redundant executions? (Section.6.3)
4. How effective is RPFUZZ in discovering new code? (Section.6.4)
5. What is RPFUZZ's capability in uncovering vulnerabilities? (Section.6.5)

Benchmark. The evaluation uses benchmark provided by ProFuzzBench (Natella and Pham (2021)). This is currently the standard benchmark for all CGF research on network service software.

Baseline. To evaluate the effectiveness of our method, we utilized the original NYX-NET as the baseline.

Experiment Environment. All experiments were conducted in Docker on a Ubuntu 22.04 LTS server, with 408 GiB of RAM on AMD® EPYC® 9654 96-Cores Processors.

6.1. Accuracy of service-related Variable Identification

As precise service-related variable(SRV) identification forms the foundation for mutation redundancy detection, this experiment evaluates our method's capability to extract SRVs through manual validation against annotations in 12 protocol implementations.

The analysis results across 12 network service implementations (Table 1) demonstrate our method's capability to identify service related variables:

- *Identification Validation.* Sampled SRVs like OpenSSL's `st.hand_state` (TLS handshake state) and tinydtls's `role` (client/server identifier) directly correspond to protocol specifications, validating the effectiveness of our filtering rules. However, depending on the software scale, residual false positives still exist. These false positives primarily stem from configuration options in service runtime environments (e.g., ProFTPD's `have_fake_mode`). Since the values of these configuration variables exhibit minimal variation during fuzzing phases, we believe that such invariability will not compromise the accuracy of the redundancy oracle. For example, if a configuration variable remains constant throughout the fuzzing process, it is unlikely to affect the detection of redundant executions.

- *Performance Analysis.* The time consumed for the whole analysis increases with the complexity of the target service. It should be noted that SRV extraction consumes less than or equal to 15% of the total analysis time (e.g., 93.47s vs 447.6s in dcmtk), proving that the main time cost comes from VFG construction. Our extraction algorithm can effectively extract SRVs in real complex network services, ensuring that the SRV identification process does not significantly impact the overall analysis efficiency.

In conclusion, the SRV extraction approach achieves accurate identification of service-related variables (SRVs) within a reasonable time, although false positives still persist. The method is effective in correlating SRV identification with protocol complexity and has a relatively low time cost compared to the overall analysis process. However, further efforts are needed to reduce the number of false positives to improve the accuracy of the redundancy oracle.

6.2. SRV Instrumentation Overhead

Instrumentation on service under test inevitably introduces execution overhead. Even with the most sophisticated feedback, excessive execution overhead can reduce the efficiency of fuzzing. In this section, we evaluate the execution overhead of SRV instrumentation by comparing the throughput of targets run by NYX-NET after 1 hour, with and without SRV instrumentation, and each experiment is repeated 5 times.

Table 3: Static Analysis Results of service-related Variable (SRV) Identification Across 12 Protocol Implementations (SVFG: Sparse Value-Flow Graph). **Full Analysis Time** consists of construction of SVFG to output SRV instrumentation information. **SRV Analyse Time** only indicates time algorithm analyzing and extracting SRVs.

Target	#SVFG MemObjects	# SVFG Edges	# Identified SRVs	Full Analyse Time	SRV Analyse Time	SRV Example
bftpd	1168	29656	7	0.44s	0.058s	`state,daemonmode,userinfo_set`
dcmtk	14666	375122	61	7m27.6s	93.47s	`go_cleanup,dbCheckMoveIdentifier`
dnsmasq	4209	238370	25	4.94s	1.28s	`client_ok,auth_dns,did_bind`
exim	3782	13934	103	1m13.77s	13.91s	`continue_more,smtp_authenticated`
forked-daapd	11400	13934	515	3m23.44s	111.52s	`response_code`
kamailio	24480	3021974	83	4m13.56s	157.78s	`ksr_evrt_received_mode`
lightftp	559	13411	4	0.21s	0.058s	`FTPCONTEXT.Access`
live555	14805	399359	14	1m29.65s	1.01s	`parseSucceeded`
openssl	71782	3454466	414	>6h	>30m	`st.hand_state,s.post_handshake_auth`
ProFTPD	13832	953796	80	2m23.74s	37.40s	`have_fake_mode,shutting_down`
pureftpd	1667	62388	27	0.92s	0.21s	`loggedin,candownload`
tinydtls	1513	37270	6	0.53s	0.030s	`state, result, role`

Table 4: Execution throughput (exec/s) overhead in 1 hour for 12 targets. **#Inst** indicates number of SRV instrumentation on store statement of service related variable. **No SRV inst/SRV inst** shows different execution speed(exec/s)on different target performed by NYX-NET.

Target	#Inst	No SRV Inst	SRV Inst	Overhead
bftpd	22	1448.1	1395.0	-3.67%
dcmtk	260	1931.1	1887.0	-4.30%
dnsmasq	81	2733.1	3440.0	-8.42%
exim	526	326.5	190.4	-41.68%
forked-daapd	1180	5.1	3.3	-35.29%
kamailio	83	690.5	631.5	-8.54%
lightftp	14	2028.3	1705.2	-15.93%
live555	31	144.4	109.6	-4.21%
openssl	1298	625.0	592.9	-5.13%
ProFTPD	336	789.2	716.3	-9.52%
pureftpd	64	1695.9	1643.6	-3.08%
tinydtls	19	1450.6	1389.1	-4.24%
Avg				-11.98%

Table 4 shows the execution speed overhead introduced by SRV instrumentation:

- *Overhead Introduced.* The measured extra execution overhead ranges from 3.08% (pureftpd) to 41.68% (exim), and with a mean value of 11.98% execution overhead across valid datasets.
- *Anomaly Case Study.* The number of instrumentation shows weak correlation with execution overhead. The possible reasons are as follows:
  1. Some targets' overhead is not obvious, since they benefit from the snapshot mechanism. The instrumentation overhead is mitigated by high-speed execution snapshot restoration. This means that when a snapshot is restored, the system can quickly resume execution without having to re-execute the instrumented code from scratch, thus reducing the overall overhead.
  2. Some targets' SRV instrumentation is localized in critical execution paths where SRV instrumentation is frequently triggered (e.g., identification and modification of FTP user identity). In these cases, even a small number of instrumentation can have a significant impact on the execution speed due to their frequent execution.

In conclusion, SRV instrumentation incurs runtime overhead with mean value of 11.98% varied from 3.08% to 41.68%.

6.3. Redundant Mutation Elimination

In this section, we evaluate redundant mutation elimination effectiveness from test case concision, redundant execution reduction, and fuzzing throughout. These 3 aspects are chosen because they can comprehensively reflect the impact of redundancy elimination on the fuzzing process. We statistically demonstrate our method's capability in these aspects after 48-hour fuzzing. Each experiment is conducted 5 times.

6.3.1. Test Case Length Reduction

Under equivalent code coverage, concise test cases demonstrate a higher proportion of effective mutations generated by fuzzer. The length (i.e., number of packets in a test case) of test cases retained by fuzzer upon discovering novel code coverage serves as a proxy metric for evaluating the efficiency of redundant mutation elimination. This is because shorter test cases are more likely to contain only essential inputs that trigger new code paths, while longer test cases may include redundant or irrelevant data that do not contribute to code discovery.

Table 5: Length (i.e.number of test case's packet) of Test Case Retained by NYX-NET and RPFUZZ. **Seeds Len** indicates average seeds length in ProFuzzBench; **Max** indicates the length of longest test case retained by fuzzer; **Avg** calculates the average test case length. Each experiment is conducted 5 times

Target	Seeds Len	Nyx-Net		RPFUZZ		Concision Rate	
	Avg	Max	Avg	Max	Avg	Max	Avg
bftpd	9.1	26.9	82	23.0	65	14.7%	20.7%
dcmtk	2.5	12.5	41	15.3	43	-23.1%	-4.9%
dnsmasq	1.0	22.8	57	23.7	57	-4.0%	0.0%
exim	7.0	16.1	39	15.7	34	2.4%	12.8%
forked-daapd	58	30.5	82	27.1	81	11.0%	1.2%
kamailio	3.0	91.7	218	56.2	169	38.8%	22.5%
lightftp	7.5	51.7	161	33.3	100	35.7%	37.9%
live555	4.1	57.6	162	43.6	118	24.4%	27.2%
openssl	4	18.5	52	18.7	46	-1.3%	11.5%
ProFTPD	9.1	71.5	209	50.6	149	29.2%	28.7%
pureftpd	9.1	41.3	116	37.5	109	9.3%	6.0%
tinydtls	1	48.6	153	36.1	114	25.8%	25.5%
Avg						13.6%	15.8%

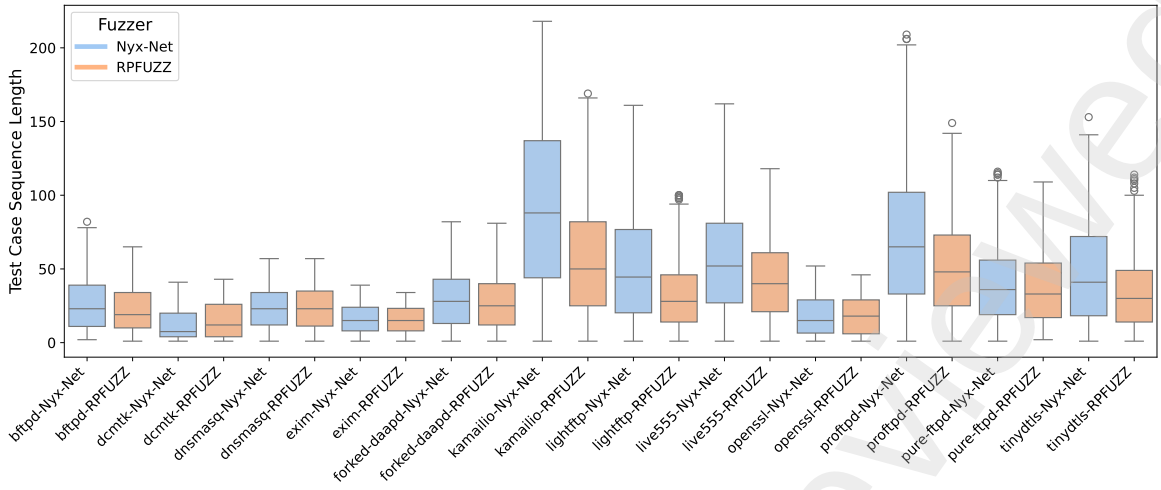

Figure 9: Distribution analysis of test case length reserved by RPFUZZ and NYX-NET. Each experiment is conducted 5 times

The comparative test case length analysis (Table 5 and Figure 9) demonstrates significant improvements in test case optimization through our redundant mutation pruning methodology:

- *Concision Performance.* As shown in Table 5, RPFUZZ demonstrated notable test case reduction: 38.8% in maximum length (kamailio) and 37.9% in average length (lightftp). Overall, it achieved average reductions of 13.6% in maximum length and 15.8% in average length. These reductions in test case length can significantly improve the efficiency of fuzzing, as shorter test cases require less time to execute and can potentially explore more code paths within the same time frame.

- *Concision Distribution.* As shown in Figure 9, compared to NYX-NET, RPFUZZ retains more concise test cases and has fewer cases where excessive redundant mutations are required to trigger new coverage. The figure clearly illustrates that RPFUZZ has a more compact distribution of test case lengths, with a higher proportion of shorter test cases. This indicates that RPFUZZ is more effective in eliminating redundant mutations and generating more efficient test cases.

- *Anomaly Case Study.* RPFUZZ failed to achieve notable test case reduction in some targets. This may be attributed to the length of initial seeds and the complexity of the targets. For instance, in dnsmasq, only one initial test case with a length of 1 was provided. Under such conditions, the test cases retained by RPFUZZ for these targets are extremely difficult to further optimize. Similarly, dcmtk provides seeds with average length of 2.5.

6.3.2. Redundant Execution Reduction

In this subsection, we evaluate the effectiveness of redundant mutation oracle by comparing the proportions of pruned execution in RPFUZZ.

We categorize the executions of RPFUZZ's pruning fuzz-routine into two types:

1. Pruned Execution (N_{Pruned}): This refers to the executions that are early terminated by the oracle during redundancy pruning fuzz. These executions are deemed redundant and are thus pruned to save computational resources.
2. Full Execution (N_{Full}): This represents the executions that complete the full fuzzing process in redundancy pruning fuzz. These executions are considered non-redundant and are used for further analysis.

We use three metrics to evaluate the performance of RPFUZZ in reducing redundant tests. These metrics help us understand the proportion of redundant executions that are pruned and the overall impact of the redundancy pruning mechanism.

The calculation of the mutation-pruned rate r_p (which indicates the proportion of test cases judged and pruned during redundancy pruning fuzz) and the pruning-fuzz rate r_f (which represents the percentage of fuzzing iterations where RPFUZZ actually executes redundancy pruning fuzz) is defined as:

$$r_p = \frac{N_{Pruned}}{N_{Pruned} + N_{Full}} \quad r_f = \frac{N_{Pruned} + N_{Full}}{N_{Pruned} + N_{Full} + N_{Regular}} \tag{4}$$

And the evaluation of effective-pruned rate r_e[3] is:

$$r_e = \frac{N_{Pruned}}{N_{Pruned} + N_{Full} + N_{Regular}} = r_p * r_f \tag{5}$$

Table 6 presents these three metrics achieved by RPFUZZ during the 48-hour fuzzing campaign:

- *Mutation-pruned Rate* (r_p) achieved by RPFUZZ demonstrates variations across different targets. Specifically, the reduction ratio ranges from a minimum of 74.31% for the complex target ProFTPD ($R_{complex} = 5.0$)[4] to a maximum of 99.46% for dcmtk ($R_{complex} = 0.66$).

[3] $N_{Rugular}$ represents the execution number of regular fuzz

[4] Complexity ratio $R_{complex}$ is calculated as N_{Edge}/N_{Node} of learned state machines by AFLNET

Table 6: RPFUZZ's mutation-pruned rate r_p, pruning-fuzz rate r_f, and effective-pruned rate r_e in 48-hour fuzzing routine. Each experiment is conducted 5 times (Targets sorted alphabetically)

Target	r_p(%)	r_f(%)	r_e(%)
bftpd	87.72	47.37	41.55
dcmtk	99.46	1.06	1.05
dnsmasq	98.95	42.78	42.33
exim	74.31	29.12	21.64
forked-daapd	86.03	35.06	30.16
kamailio	66.01	39.66	26.18
lightftp	90.85	47.34	43.01
live555	65.00	44.80	29.12
openssl	90.04	2.38	2.14
ProFTPD	72.97	45.06	32.88
pureftpd	81.43	47.95	39.05
tinydtls	93.32	49.10	45.82
Avg	83.84	35.97	29.58

- *Test Case Pruning-fuzz Rate* (r_f) exhibits notable differences among targets. It is primarily influenced by the initial test case length and complexity of the service protocols involved. Since pruning operations only occur when long suffix lengths exist, targets with predominantly short test cases (e.g., DCMTK with 60% test cases of length 1) demonstrate significantly lower r_f compared to those with longer sequences (e.g., ProFTPD with 80% test cases of length 4).

- *Effective Pruning Rate* (r_e). The combined effects of varying r_p and r_f resulted in different effective-pruned rate across targets. RPFUZZ achieves an effective redundancy reduction ranging from 1.05% (minimum) to 43.01% (maximum), demonstrating consistent optimization effectiveness across different software targets.

Figure.10 presents the temporal dynamics of mutation-pruned ratio r_p during the initial two-hour fuzzing-routine of RPFUZZ:

- *Adaptive Pruning Growth:* r_p exhibits a logarithmic growth trend over time, indicating that the problem of redundant mutations becomes more pronounced as testing progresses. This growth trend can be attributed to several factors, such as the increasing number of generated test cases and the limited exploration of new code paths as fuzzing continues. Meanwhile, the redundancy oracle dynamically identifies redundant mutations by collecting known execution contexts during fuzzing, enabling timely termination of such unnecessary executions. This adaptive pruning mechanism helps to save computational resources and improve the efficiency of fuzzing.

- *Case study.* The variation patterns of r_p across different targets demonstrate distinct characteristics. For example, targets with more complex protocol implementations may exhibit a faster initial growth rate of r_p due to the larger number of potential redundant mutations. In contrast, simpler targets may have a relatively slower growth rate. These differences in variation patterns are closely related to the complexity of the protocol implementations, as more complex protocols often involve more intricate state transitions and input handling, leading to a higher likelihood of redundant mutations.

6.3.3. Throughput Improvement

Early termination of redundant mutated test cases will directly influence the fuzzer's throughput. Higher throughput enables dynamic redistribution of computational resources toward test cases exhibiting novel path-discovering potential.

In this section, we examine RPFUZZ by measuring fuzzing throughput in a 48hour fuzz-routine as shown in Table 7:

- *Throughput Performance*. RPFUZZ demonstrates substantial performance gains over NYX-NET, with 185.9% (exim) maximum speedup and 59.2% mean acceleration in test execution. These improvements are enabled by its real-time redundancy detection mechanism. This mechanism continuously monitors the execution of test cases and identifies redundant mutations by comparing their execution contexts with those of previously executed cases. Once a redundant mutation is detected, it is terminated early, thus saving computational resources and increasing the overall throughput.

- *Anomaly Case Study*. RPFUZZ maintained the fuzzing throughput for DCMTK without observable degradation. This phenomenon can be attributed to the combined effects of DCMTK's exceptionally short initial seeds (average length = 2.5) and the operational characteristics of RPFUZZ. While achieving high test case reduction rates, the infrequent activation of pruning-aware fuzzing phases (low r_f = 1.06%) amplified the relative impact of SRV instrumentation overhead. Specifically, the runtime costs introduced by instrumentation disproportionately affected the efficiency of regular fuzzing cycles. Since the pruning opportunities were limited due to the short initial seeds and low r_f, the persistent instrumentation burden across predominantly non-pruning test iterations could not be offset, leading to a situation where the efficiency of regular fuzzing was not significantly improved despite the high test case reduction rates.

Table 7: Final execution throughout (exec/s) in 48-hour fuzzing routines performed by NYX-NET and RPFUZZ. Each experiment is conducted 5 times

Target	Nyx-net	RPFUZZ	Improvement
bftpd	1377.0	1574.0	+14.3%
dcmtk	1931.1	1887.0	-8.8%
dnsmasq	2733.1	3440.0	+25.8%
exim	326.2	932.6	+185.9%
forked-daapd	5.6	9.8	+75.2%
kamailio	653.8	1219.1	+86.4%
lightftp	1787.6	2729.5	+53.2%
live555	53.9	87.2	+61.7%
openssl	672.2	758.1	+12.8%
ProFTPD	727.1	1056.5	+45.3%
pureftpd	1720.1	2241.8	+30.3%
tinydtls	1108.6	2526.1	+127.8%
Avg			+59.2%

In conclusion, RPFUZZ demonstrates remarkable performance improvements. It achieves an effective pruning rate

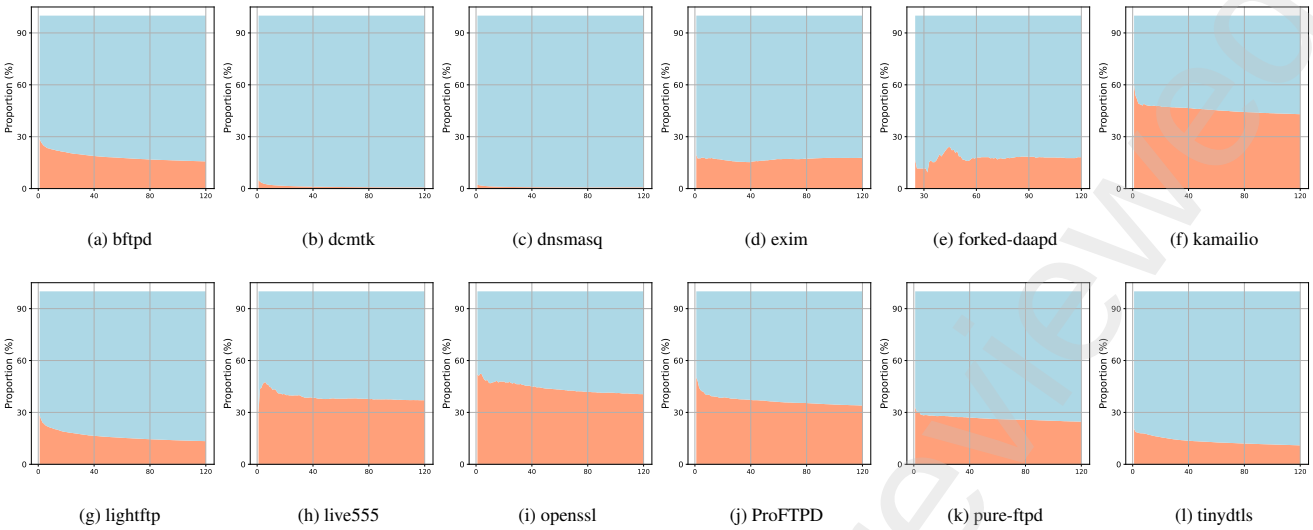

Figure 10: Proportions of Pruned Execution and Full Execution (%) Over Time (min) in 2 hours fuzzing

of more than 45.82% (with an average of 29.58%), a maximum throughput enhancement of 185.92% (with an average of 59.2%), and a reduction in test case length of 37.9% (with an average of 15.8%) compared to the baseline method. Moreover, it maintains full coverage integrity throughout the fuzzing process and effectively minimizes redundant test executions.

6.4. Coverage Exploration Capability

Code coverage remains the gold standard for fuzzing effectiveness. We compare code coverage with NYX-NET to validate our method's code discovery capability in complex stateful services in Table.8:

Table 8: Code Coverage Comparison (Bitmap) between NYX-Net and RPFUZZ. Each experiment is conducted 5 times

Target	NYX-NET	RPFUZZ	Improvement Rate
bftpd	693.2	705	+1.70%
dcmtk	6314.4	6584.2	+4.27%
dnsmasq	793.2	788.2	-0.63%
exim	3549.0	3573.0	+0.68%
forked-daapd	3144.0	3209.0	+2.07%
kamailio	10194.2	10602.2	+4.00%
lightftp	804.8	797.6	-0.89%
live555	5023.8	5085.2	+1.22%
openssl	12204.4	12264.8	+0.49%
ProFTPD	6661.2	6611.6	-0.74%
pureftpd	1649.0	1652.4	+0.21%
tinydtls	848.0	851.0	+0.35%
Avg			+1.02%

- RPFUZZ achieves a +1.02% average code coverage improvement over NYX-NET across 12 targets, with particularly significant gains in dcmtk (+4.27%) and kamailio (+4.0%). This improvement in code coverage is crucial as it indicates that RPFUZZ may be able to discover more vulnerabilities hidden in the previously uncovered code.
- Seven targets show minor improvements (+0.21% to +1.22%), while three exhibit slight regression (-0.63% to -0.89%), demonstrating RPFUZZ's overall effectiveness in maintaining coverage fidelity during aggressive test case reduction.

RPFUZZ demonstrates a +1.02% average code coverage improvement over baseline methods, with improvements observed in 8 out of 12 targets (range: +0.21% to +4.27%). This shows RPFUZZ's systematic approach to reduce redundant mutation without compromising exploration capability. RPFUZZ achieves this balance by accurately identifying and pruning redundant mutations while employing intelligent test case generation strategies to ensure that the code space is thoroughly explored.

6.5. Vulnerability Discovery

The ultimate security value of fuzzing is uncovering vulnerabilities. We have documented the crashes found by RPFUZZ along with their discovery time, including the assigned CVE. This demonstrates RPFUZZ's capability in vulnerability discovery through systematic fuzzing.

Table 9: Overview of crash discovery statuses in ProfuzzBench during 48 hour fuzzing

Target	Crash	TTE
bftpd	-	-
dcmtk	✓	28h 59m 56s
dnsmasq	✓	2m 0s
exim	✓	13h 47m 22s
forked-daapd	-	-
kamailio	-	-
lightftp	-	-
live555	✓	34s
openssl	-	-
ProFTPD	✓	1h 52m 27s
pureftpd	-	-
tinydtls	✓	55s

Figure 9 illustrates crash discovery status and time-to-exposure (TTE) for targets tested by RPFUZZ without ASAN

(sanitizer for memory flaw detection). Among 6 network service targets flagged by RPFUZZ for potential vulnerabilities, manual analysis confirmed that most crashes corresponded to known 1-day exploits or documented implementation flaws. Notably, preliminary manual verification of CVE-2024-57392 — a buffer boundary violation in ProFTPD—revealed that malformed `STAT` commands from FTP clients trigger an out-of-bounds memory write during `pr_data_xfer` operations via unsafe `memcpy` usage. This vulnerability allows attackers to remotely trigger denial-of-service (DoS) conditions on ProFTPD instances with vulnerable configurations.

In conclusion, RPFUZZ successfully identified flaws across six targets without ASAN and discovered a buffer overflow vulnerability in ProFTPD (assigned CVE-2024-57392). This demonstrates RPFUZZ's capability in vulnerability discovery.

7. Related Work

AFLNET (Pham et al. (2020)) is the first CGF framework using code coverage and response code feedback to guide network service fuzzing. Recent works have attempted to address the unique challenges of CGF from several aspects:

To improve AFLNET's throughput, NSFUZZ employed modified packet-synchronization mechanism (Qin et al. (2023)) to enhance the efficiency of test case delivery. This mechanism addresses the issue where AFLNET is unable to promptly determine when the target under test has finished processing a test case. NYX-NET(Schumilo et al. (2022)) and SNPSFUZZER(Li et al. (2022)) employ VM incremental snapshots and CRIU (Checkpoint/Restore In Userspace) (CRIU (2025)) snapshots respectively, to reduce the overhead introduced by sending M1 messages. HNPFUZZER(Fu et al. (2023)) applied network I/O simulation methods by leveraging fast share-memory I/O like popular desocket tool libpreeny(Shoshitaishvili (2025)). In this work, we present an orthogonal approach to improve the efficiency of fuzz testing by timely terminating redundant mutation tests. Our method can be combined with these methods that excel in direct throughput to further enhance the efficiency of fuzz testing.

While response code is not accurate in state description in AFLNET, other work introduce another metric like long-lived memory (Natella (2021)), and state variables (Qin et al. (2023); Ba et al. (2022)). In our work, the definition and recognition method of service related variable(SRV) is different from these works. Moreover, we utilize SRV to construct a more precise execution context, enabling the redundancy oracle to make more accurate judgments.

For better test case selection, AFLNET-Legion (Liu et al. (2022)) further investigates using Monte Carlo tree to improve AFLNET's state selection algorithm. However, this method has demonstrated a slight improvement in code coverage.

8. Conclusion

In this paper, we have successfully addressed the issue of redundant mutation in network service fuzzing through the introduction of a mutation-redundancy oracle. By employing three key technologies, namely (1) the identification of service related variables (SRVs) via value flow graph analysis, (2) a redundant mutation oracle powered by SRV-enhanced execution context, and (3) an adaptive sequence pruning method for network service fuzzing. The empirical results obtained from ProFuzzBench demonstrate the practical efficacy of our approach, showcasing a notable 185.92%(average 59.2%) boost in test throughput, 4.27% more code coverage (average 1.02%), and uncovering 1 vulnerability in real-world applications.

Looking ahead, there are several optimization strategies that could further enhance the efficiency of redundancy-pruning fuzzing. Firstly, we can explore an instrumentation strategy decoupling approach. During the standard fuzzing phase, only code coverage instrumentation is employed, while SRV instrumentation is reserved specifically for redundancy-pruning fuzzing. This approach has the potential to alleviate performance overheads because it reduces the unnecessary instrumentation during the regular fuzzing cycles, thus improving the overall efficiency. Secondly, dynamic parameter adaptation based on real-time monitoring of redundancy and exploration probabilities offers great promise. By continuously monitoring these probabilities, we can fine-tune the framework's parameters in real-time. For example, if the redundancy probability is high, we can adjust the fuzzing strategy to focus more on reducing redundancy, and vice versa. This dynamic adjustment can lead to enhanced performance of the fuzzing framework.

Through the implementation of these two potential improvements, the redundancy-pruning technology is expected to mature further. This will significantly enhance the efficiency of fuzz testing for network service software in the future, enabling more vulnerabilities to be discovered in a shorter period of time and with fewer computational resources.

References

Aschermann, C., Schumilo, S., Abbasi, A., Holz, T., 2020. Ijon: Exploring deep state spaces via fuzzing, in: 2020 IEEE Symposium on Security and Privacy (SP), pp. 1597–1612. doi:10.1109/SP40000.2020.00117.

Ba, J., Böhme, M., Mirzamomen, Z., Roychoudhury, A., 2022. Stateful Greybox Fuzzing. URL: `http://arxiv.org/abs/2204.02545`. arXiv:2204.02545 [cs].

boofuzz, 2024. boofuzz: Network Protocol Fuzzing for Humans — boofuzz 0.4.2 documentation. URL: `https://boofuzz.readthedocs.io/en/stable/index.html`.

Chen, H., Xue, Y., Li, Y., Chen, B., Xie, X., Wu, X., Liu, Y., 2018. Hawkeye: Towards a desired directed grey-box fuzzer, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, Association for Computing Machinery, New York, NY, USA. p. 2095–2108. URL: `https://doi.org/10.1145/3243734.3243849`, doi:10.1145/3243734.3243849.

Chen, Y., Li, P., Xu, J., Guo, S., Zhou, R., Zhang, Y., Wei, T., Lu, L., 2020. SAVIOR: Towards Bug-Driven Hybrid Testing, in: 2020 IEEE Symposium on Security and Privacy (SP), pp. 1580–1596. URL: `https://ieeexplore.ieee.org/abstract/document/9152682`, doi:10.1109/SP40000.2020.00002. iSSN: 2375-1207.

CRIU, 2025. checkpoint-restore/criu. URL: `https://github.com/checkpoint-restore/criu`. original-date: 2014-01-17T12:53:25Z.

Dymarski, P. (Ed.), 2011. Hidden Markov Models: Theory and Applications. IntechOpen, Erscheinungsort nicht ermittelbar.

Fioraldi, A., D'Elia, D.C., Balzarotti, D., 2021. The use of likely invariants as feedback for fuzzers, in: 30th USENIX Security Symposium (USENIX Security 21), pp. 2829–2846.

Fioraldi, A., Maier, D., Eißfeldt, H., Heuse, M., 2020. {AFL++}: Combining incremental steps of fuzzing research, in: 14th USENIX workshop on offensive technologies (WOOT 20).

Fu, J., Xiong, S., Wang, N., Ren, R., Zhou, A., Bhargava, B.K., 2023. A Framework of High-Speed Network Protocol Fuzzing Based on Shared Memory. IEEE Transactions on Dependable and Secure Computing , 1–18URL: https://ieeexplore.ieee.org/document/10262045/, doi:10.1109/TDSC.2023.3318571.

Gascon, H., Wressnegger, C., Yamaguchi, F., Arp, D., Rieck, K., 2015. Pulsar: Stateful Black-Box Fuzzing of Proprietary Network Protocols, in: Thuraisingham, B., Wang, X., Yegneswaran, V. (Eds.), Security and Privacy in Communication Networks, Springer International Publishing, Cham. pp. 330–347.

Lei, Y., Sui, Y., 2019. Fast and precise handling of positive weight cycles for field-sensitive pointer analysis, in: Static Analysis: 26th International Symposium, SAS 2019, Porto, Portugal, October 8–11, 2019, Proceedings 26, Springer. pp. 27–47.

Li, J., Li, S., Sun, G., Chen, T., Yu, H., 2022. SNPSFuzzer: A Fast Greybox Fuzzer for Stateful Network Protocols using Snapshots. URL: http://arxiv.org/abs/2202.03643. arXiv:2202.03643 [cs].

Li, P., Meng, W., Zhang, C., 2024. {SDFuzz}: Target states driven directed fuzzing, in: 33rd USENIX Security Symposium (USENIX Security 24), pp. 2441–2457.

libfuzzer, 2025. fuzzing/tutorial/libFuzzerTutorial.md at master · google/fuzzing. URL: https://github.com/google/fuzzing/blob/master/tutorial/libFuzzerTutorial.md.

Liu, D., Pham, V.T., Ernst, G., Murray, T., Rubinstein, B.I., 2022. State Selection Algorithms and Their Impact on The Performance of Stateful Network Protocol Fuzzing, in: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, Honolulu, HI, USA. pp. 720–730. URL: https://ieeexplore.ieee.org/document/9825758/, doi:10.1109/SANER53432.2022.00089.

antonio morales, 2020. Fuzzing sockets, part 1: FTP servers. URL: https://securitylab.github.com/resources/fuzzing-sockets-FTP/.

Nagy, S., Hicks, M., 2019. Full-speed fuzzing: Reducing fuzzing overhead through coverage-guided tracing, in: 2019 IEEE Symposium on Security and Privacy (SP), pp. 787–802. doi:10.1109/SP.2019.00069.

Natella, R., 2021. StateAFL: Greybox Fuzzing for Stateful Network Servers. URL: http://arxiv.org/abs/2110.06253. arXiv:2110.06253 [cs].

Natella, R., Pham, V.T., 2021. ProFuzzBench: a benchmark for stateful protocol fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, ACM, Virtual Denmark. pp. 662–665. URL: https://dl.acm.org/doi/10.1145/3460319.3469077, doi:10.1145/3460319.3469077.

NVD, 2025a. NVD - CVE-2014-0160. URL: https://nvd.nist.gov/vuln/detail/CVE-2014-0160.

NVD, 2025b. NVD - CVE-2020-0796. URL: https://nvd.nist.gov/vuln/detail/CVE-2020-0796.

peach, 2025. Peach by MozillaSecurity. URL: https://mozillasecurity.github.io/peach/.

Pham, V.T., Bohme, M., Roychoudhury, A., 2020. AFLNET: A Greybox Fuzzer for Network Protocols, in: 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), IEEE, Porto, Portugal. pp. 460–465. URL: https://ieeexplore.ieee.org/document/9159093/, doi:10.1109/ICST46399.2020.00062.

Qin, S., Hu, F., Ma, Z., Zhao, B., Yin, T., Zhang, C., 2023. Nsfuzz: Towards efficient and state-aware network service fuzzing. ACM Transactions on Software Engineering and Methodology 32, 1–26.

Schumilo, S., Aschermann, C., Jemmett, A., Abbasi, A., Holz, T., 2022. Nyx-net: network fuzzing with incremental snapshots, in: Proceedings of the Seventeenth European Conference on Computer Systems, ACM, Rennes France. pp. 166–180. URL: https://dl.acm.org/doi/10.1145/3492321.3519591, doi:10.1145/3492321.3519591.

Shoshitaishvili, Y., 2025. zardus/preeny. URL: https://github.com/zardus/preeny. original-date: 2015-03-15T23:32:45Z.

crowd strike, 2025. 2025 Global Threat Report | Latest Cybersecurity Trends & Insights | CrowdStrike. URL: https://www.crowdstrike.com/en-us/global-threat-report/.

Sui, Y., Ye, D., Xue, J., 2014. Detecting Memory Leaks Statically with Full-Sparse Value-Flow Analysis. IEEE Transactions on Software Engineering 40, 107–122. URL: https://ieeexplore.ieee.org/abstract/document/6720116, doi:10.1109/TSE.2014.2302311. conference Name: IEEE Transactions on Software Engineering.

SVF-tools, 2025. SVF-tools/SVF. URL: https://github.com/SVF-tools/SVF. original-date: 2015-06-05T01:52:24Z.

Whalen, S., Bishop, M., Crutchfield, J.P., 2010. Hidden markov models for automated protocol learning, in: Security and Privacy in Communication Networks: 6th Iternational ICST Conference, SecureComm 2010, Singapore, September 7-9, 2010. Proceedings 6, Springer. pp. 415–428.

Yue, T., Wang, P., Tang, Y., Wang, E., Yu, B., Lu, K., Zhou, X., 2020. {EcoFuzz}: Adaptive {Energy-Saving} greybox fuzzing as a variant of the adversarial {Multi-Armed} bandit, in: 29th USENIX Security Symposium (USENIX Security 20), pp. 2307–2324.